

WORKINGWORKING 2

January 9, 2026

```
[1]: '''Work out what are my issues with this code before seeing gavin.
      Get a standard event with all info included then run code and see how it
      ↳behaves.
      From this Ill be able to identify where my code is not working e.g. with the
      ↳time vs observe time, single frequency vs multiple
      '''
```

```
[1]: 'Work out what are my issues with this code before seeing gavin. \nGet a
      standard event with all info included then run code and see how it behaves.
      \nFrom this Ill be able to identify where my code is not working e.g. with the
      time vs observe time, single frequency vs multiple \n'
```

```
[2]: import redback
      print(redback.__version__)
```

```
No module named 'lalsimulation'
lalsimulation is not installed. Some EOS based models will not work. Please use
bilby eos or pass your own EOS generation class to the model
12:51 bilby INFO      : Running bilby version: 2.3.0
12:51 redback INFO    : Running redback version: 1.12.1
1.12.1
```

```
[3]: import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
      import redback.interaction_processes as ip
      import redback.sed as sed
      import redback.photosphere as photosphere
      from astropy.cosmology import Planck18 as cosmo
      import astropy.units as uu

      import extinction
      from extinction import ccm89, fitzpatrick99, apply, remove
      from scipy.interpolate import RegularGridInterpolator
      import sncosmo
```

```

[12]: '''GRB EVENT ANALYSIS:'''
#STEP 1: Load in GRB event information eg. AB magnitude, redshift, filter,
↳frequency of filter?, Av if applicable

#GRB: 100206A has mag = 21.7 , z = 0.4068 at 0.1315 days in R band
#GRB 060218 has mag_AB = 17.22, epoch = 11.0, z_event = 0.0331 in R-band, f_98
↳= 0.7 -- THIS IS WHAT WE ARE TRYING TO ACHIEVE -- DID WE ACHIEVE IT ? --
#where did i get this information from ?

#EVENT I KNOW ALL THE INFO FOR AND A CLEAR RESULT FOR F_98 :
#GRB 980425, associated with 1998bw so f_98 = 1
#z = 0.085,

#USE A DIFFERENT WELL KNOWN ONE
# GRB 171205A , z = 0.0368,
# epoch = 10.98015 mag = 17.7, band = I
#E_(B-V)= 0.05

event_name = '171205A'
AB_mag = 17.7
redshift = 0.0368
epoch = 10.98
band = 'besselli'
a_v = 0.05 * 3.1 #E(B-V) x R_v

#WHAT IS GOING ON WITH THE FREQUENCIES ?

#STEP 2: Convert GRB event AB magnitude into flux density using redback
def calc_flux_density_from_ABmag(AB_mag):
    """
    Calculate flux density from AB magnitude assuming monochromatic AB filter

    :param magnitudes: AB magnitude values
    :return: flux density
    """
    return (AB_mag * uu.ABmag).to(uu.mJy)

flux_density_event = calc_flux_density_from_ABmag(AB_mag)

print("This is the flux density of GRB event", event_name ,f"without
↳extinction correction: {flux_density_event:.3f}")

#check this using online calculator -- CORRECT -- OK TO PROCEED TO NEXT STEP

#STEP 3: Use fitzpatrick99 extinction function (if applicable) to find
↳de-reddened value for GRB flux density
#to get wavelength for GRB event, use redback tables

```

```

#wavelength must be an array
wavelength = np.array([8020.14000]) #angstroms -- i-band wavelength
dereddened_flux_event = extinction.remove(fitzpatrick99(wavelength, a_v, 3.
    ↪1),flux_density_event)

#dereddened flux is now a 1-d array so display the first element
print(f"This is the extinction corrected GRB event flux density found using
    ↪fitzpatrick99: {dereddened_flux_event[0]:.3f}")

#to test if this is doing what it should be doing, if i increase the a_v value,
    ↪the flux should go upwards!
# if Av is changed to be higher 4-5 than the original value, flux density
    ↪should be getting brighter !!
#DOES THIS HAPPEN ? -- YES -- the value goes up !!

#PROCEED TO NEXT STEP

'''SN1998BW MODEL ANALYSIS:'''
#STEP 4: Define band from sncosmo to match the filter for the GRB event with
    ↪the frequency bandpass filter for input into SN1998bw model

print("GRB",event_name, "is associated with the filter band", band)

#we need to get frequency bandpass for the specific filter using sncosmo
bandpass = sncosmo.get_bandpass(band)
# wavelengths (in angstroms)
print(bandpass.wave)
#convert wavelength to m
wavelength_m = bandpass.wave *1e-10
#define c
c = 3.0e8
#now to get in terms of frequencies have to calculate using  $f = c / \text{wavelength}$ 
frequencies_Hz = c / wavelength_m
#check + also do by hand -- DOES THIS WORK ? -- YES
print(frequencies_Hz)

#STEP 5:

#calculate 1-D array of flux densities of 1998bw over time
#bug fix : ensure time is the same or related to the observed time
# i was working in two different frames which caused a 0 error

#define lambda_to_nu outside of the function: does this make the previous
    ↪frequencies_Hz obsolete ?

```

```

def lambda_to_nu(wavelength_angstrom):
    """ Converts wavelength in Angstroms to frequency in Hz """
    c = 299792458 # speed of light in m/s
    return c / (wavelength_angstrom * 1e-10)

#change the inner workings of the sn1998bw def
def sn1998bw_template(time, redshift, amplitude, **kwargs):
    """
        A wrapper to the SN1998bw template. Only valid between 1100-11000 Angstrom
        and 0.01 to 90 days post explosion in rest frame

        Parameters
        -----
        time
            time in days in observer frame (post explosion)
        redshift
            redshift
        amplitude
            amplitude scaling factor, where 1.0 is the original brightness of
            SN1998bw; and f_lambda is scaled by this factor
        kwargs
            Additional keyword arguments required by redback.
        frequency
            Required if output_format is 'flux_density'. frequency to calculate -
            Must be same length as time array or a single number).
        bands
            Required if output_format is 'magnitude' or 'flux'.
        output_format
            'flux_density', 'magnitude', 'spectra', 'flux', 'sncosmo_source'
        cosmology
            Cosmology to use for luminosity distance calculation. Defaults to
            Planck18. Must be a astropy.cosmology object.
        Returns
        -----
            set by output format - 'flux_density', 'magnitude', 'spectra', 'flux',
            'sncosmo_source'

    """
    import sncosmo
    model = sncosmo.Model(source='v19-1998bw')
    original_redshift = 0.0085 #redshift of 1998bw
    cosmology = kwargs.get("cosmology", cosmo)
    original_dl = (43*uu.Mpc).to(uu.cm).value

    # From roughly matching to Galama+ or Clocchiatti+1998bw light curves
    original_peak_time = 15
    model.set(z=original_redshift, t0=original_peak_time)

```

```

model.set_source_peakmag(14.25, band='bessellb', magsys='ab')
lls = np.linspace(1620, 11000, 300)
f_lambda = model.flux(time, lls) #erg/s/cm^2/Angstrom.
l_lambda = f_lambda * 4 * np.pi * original_dl**2 # erg/s/Angstrom

# We consider this the rest frame spectrum of 1998bw. Now we can redshift
↳ it and scale it.
time_obs = time * (1 + redshift)
lambda_obs = lls * (1 + redshift)
dl_new = cosmology.luminosity_distance(redshift).cgs.value
f_lambda_obs = l_lambda / (4 * np.pi * dl_new**2)
f_lambda_obs = amplitude * f_lambda_obs * (1 + redshift) # accounting for
↳ bandwidth stretching
f_lambda_obs = f_lambda_obs * uu.erg / uu.s / uu.cm ** 2 / uu.Angstrom
if kwargs['output_format'] == 'flux_density':
    frequency = kwargs['frequency']
    # work in obs frame
    ff_array = lambda_to_nu(lambda_obs)

    # Convert flux density to mJy
    fmjy = f_lambda_obs.to(uu.mJy, equivalencies=uu.
↳ spectral_density(wav=lambda_obs * uu.Angstrom)).value
    # Create interpolator on obs frame grid
    flux_interpolator = RegularGridInterpolator(
        (time_obs, ff_array),
        fmjy,
        bounds_error=False,
        fill_value=0.0)

    # Prepare points for interpolation
    if isinstance(frequency, (int, float)):
        frequency = np.ones_like(time) * frequency

    # Create points for evaluation
    #MODIFIED !!
    #points = np.column_stack((time, frequency))
    # FIX: Align the query points with the grid (time_obs)
    points = np.column_stack((time_obs, frequency))

    # Return interpolated flux density with (1+z) correction for observer
↳ frame
    return flux_interpolator(points)
    elif kwargs['output_format'] == 'spectra':
        return namedtuple('output', ['time', 'lambdas',
↳ 'spectra'])(time=time_obs,

↳ lambdas=lambda_obs,

```

```

↪spectra=f_lambda_obs)
    else:
        return sed.get_correct_output_format_from_spectra(time=time,
↪time_eval=time_obs,
                                                                    spectra=f_lambda_obs,
↪lambda_array=lambda_obs,
                                                                    **kwargs)

# Just pass an array of two values to satisfy the interpolator's grid
↪requirement
target_epoch = epoch
times = np.array([target_epoch, target_epoch + 0.1])

#only takes in one specific frequency -- so what was the point in the sncosmo
↪bandpass ?!!!!!! WHAT WAS IT ? -- DO WE STILL NEED IT --

result = sn1998bw_template(
    time=times,
    redshift=redshift,
    amplitude=1.0,
    output_format='flux_density',
    frequency=6.76322e+13, #singular frequency -- CORRECT ? from redback tables
↪I-band effective width in Hz
    cosmology=cosmo
)

f_1998bw_interpolated = result[0]

# result[0] is now exactly the flux at day 10.5
print(f"Flux density at day {target_epoch}: {f_1998bw_interpolated} mJy")

#SKIPPED STEP 6 -- INTERPOLATED IN ONE GO

'''FINAL RATIO:'''
#STEP 7: take value from STEP 3 and STEP 6 in ratio with one another to get
↪final result of flux ratio

f_1998bw_ratio = dereddened_flux_event / f_1998bw_interpolated

print(f"The final flux density ratio result of F_GRB / F_1998bw =
↪{f_1998bw_ratio[0]:.3f}")

```

```
#RETURNS A ZERO ERROR - CHANGE THE POINTS ON TIME -- MAKES A DIFERENCE ?
```

This is the flux density of GRB event 171205A without extinction correction:
0.302 mJy

This is the extinction corrected GRB event flux density found using
fitzpatrick99: 0.327 mJy

GRB 171205A is associated with the filter band besselli

```
[7000. 7100. 7200. 7300. 7400. 7500. 7600. 7700. 7800. 7900. 8000. 8100.  
 8200. 8300. 8400. 8500. 8600. 8700. 8800. 8900. 9000. 9100. 9200.]
```

```
[4.28571429e+14 4.22535211e+14 4.16666667e+14 4.10958904e+14  
 4.05405405e+14 4.00000000e+14 3.94736842e+14 3.89610390e+14  
 3.84615385e+14 3.79746835e+14 3.75000000e+14 3.70370370e+14  
 3.65853659e+14 3.61445783e+14 3.57142857e+14 3.52941176e+14  
 3.48837209e+14 3.44827586e+14 3.40909091e+14 3.37078652e+14  
 3.33333333e+14 3.29670330e+14 3.26086957e+14]
```

Flux density at day 10.98: 0.0 mJy

The final flux density ratio result of $F_{\text{GRB}} / F_{1998\text{bw}} = \text{inf}$ mJy

[5]: *#CODE OPTIMIZED BY AI:*

```
#main issues noted were: frequency disconnect, time vs time obs with the 1+z  
↪shift, redundant constants
```

```
import numpy as np  
import astropy.units as uu  
from astropy.cosmology import Planck18 as cosmo  
from astropy.constants import c  
import sncosmo  
import extinction  
from extinction import fitzpatrick99  
from scipy.interpolate import RegularGridInterpolator
```

```
# --- STEP 1: Event Data ---
```

```
event_name = '060218'  
AB_mag = 17.22  
redshift = 0.0331  
epoch_obs = 11.0 # Days in observer frame  
band = 'bessellr'  
a_v = 0.39 #true value from literature
```

```
#define amplitude -- makes no difference still factor 10 out  
amplitude = 1.0
```

```
# --- STEP 2 & 3: Flux & Extinction ---
```

```
def get_dereddened_flux(mag, av, band_name):  
    # 1. Convert Mag to mJy
```

```

flux_raw = (mag * uu.ABmag).to(uu.mJy)

# 2. Get central wavelength for extinction
bp = sncosmo.get_bandpass(band_name)
wave_eff = np.array([bp.wave_eff]) # Angstroms

# 3. Apply Extinction (Note: fitzpatrick99 expects wave in Angstroms)
# extinction.remove returns the de-reddened flux
flux_corr = extinction.remove(fitzpatrick99(wave_eff, av, 3.1), [flux_raw.
↪value])[0]
    return flux_corr * uu.mJy, bp.wave_eff

flux_event_corr, wave_eff = get_dereddened_flux(AB_mag, a_v, band)
print(f"Dereddened Flux ({event_name}): {flux_event_corr:.3f}")

# --- STEP 4 & 5: SN1998bw Template ---

def get_sn1998bw_flux(time_obs, z_event, band_name):
    # Initialize the specific 1998bw template
    model = sncosmo.Model(source='v19-1998bw')

    # 1. ANCHOR THE MODEL
    # We set the peak magnitude to what SN1998bw actually was
    # (Approx -19.29 absolute mag, or 14.25 apparent at its actual distance)
    # This ensures 'amplitude' is set to a physical value.
    model.set(z=z_event)
    model.set_source_peakmag(14.25, 'bessellb', 'ab')

    # 2. CALCULATE MAGNITUDE
    # Calculate the apparent magnitude at the observer's time
    try:
        mag_at_epoch = model.bandmag(band_name, 'ab', time_obs)

        # 3. CONVERT TO mJy
        # Using the standard AB flux density formula
        flux_mJy = (mag_at_epoch * uu.ABmag).to(uu.mJy)
        return flux_mJy
    except ValueError:
        # This happens if the time_obs is outside the template range (0-90 days)
        print(f"Warning: Time {time_obs} is out of bounds for the template.")
        return 0 * uu.mJy

# Re-run the calculation
f_1998bw = get_sn1998bw_flux(epoch_obs, redshift, band)
print(f"Corrected SN1998bw flux at day {epoch_obs}: {f_1998bw:.3f}")

ratio = flux_event_corr / f_1998bw

```



```

print(f"Corrected Final Flux Ratio: {ratio.value:.3f}")

#now only order of 10 out !! -- to do with a_v ? return to correct number,
↳still factor of 10 too small

#beter than it wss !

```

Dereddened Flux (060218): 0.619 mJy
Corrected SN1998bw flux at day 11.0: 7.156 mJy
Corrected Final Flux Ratio: 0.086

```

[7]: import matplotlib.pyplot as plt

def plot_grb_analysis(time_obs, flux_event, z_event, band_name):
    # 1. Setup the model
    model = sncosmo.Model(source='v19-1998bw')
    model.set(z=z_event)
    model.set_source_peakmag(14.25, 'bessellb', 'ab')

    # 2. Generate time array for the curve (0 to 80 days post-explosion)
    t_range = np.linspace(0.1, 80, 200)

    # 3. Calculate model fluxes in mJy
    model_fluxes = []
    for t in t_range:
        m = model.bandmag(band_name, 'ab', t)
        f = (m * uu.ABmag).to(uu.mJy).value
        model_fluxes.append(f)

    # 4. Plotting
    plt.figure(figsize=(10, 6))
    plt.plot(t_range, model_fluxes, label=f'SN1998bw Template (z={z_event})',
↳color='royalblue', lw=2)
    plt.scatter(time_obs, flux_event.value, color='red', s=100, label=f'GRB_
↳{event_name} Data (Dereddened)', zorder=5)

    plt.xlabel('Time since explosion (days)', fontsize=12)
    plt.ylabel('Flux Density (mJy)', fontsize=12)
    plt.title(f'Comparison: GRB {event_name} vs SN1998bw Template ({band_name}_
↳band)', fontsize=14)
    plt.grid(alpha=0.3)
    plt.legend()
    plt.show()

# Execute the plot
plot_grb_analysis(epoch_obs, flux_event_corr, redshift, band)

```

#WELL THIS LOOKS INTERESTING -- AND WRONG !! SHOW TO GAVIN TOMRROW EITHER VIA  EMAIL OR IN PERSON - THEN EMAIL JILLIAN AND ANDREW

